Constructor Overloading

Constructor Overloading in Java

Constructor overloading occurs when a class has more than one constructor, each with a different parameter list. This allows you to create objects in different ways, providing flexibility in object initialization.

Code Example - Constructor Overloading:

Let's create a `Car` class that demonstrates constructor overloading. We will have three constructors:

1. A **default constructor** that initializes the car with default values.
2. A **constructor with two parameters** for `make` and `model`, with a default `year`.
3. A **constructor with three parameters** to initialize `make`, `model`, and `year`.

Code:

```
public class Car {
    // Instance variables
    String make;
    String model;
    int year;

    // Default Constructor (No parameters)
    public Car() {
        this.make = "Unknown";
        this.model = "Unknown";
        this.year = 0; // Default year
    }

    // Constructor with two parameters (make and model)
    public Car(String make, String model) {
        this.make = make;
        this.model = model;
        this.year = 2020; // Default year
    }

    // Constructor with three parameters (make, model, and year)
    public Car(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }
```

```
    // Method to display car details
    public void display() {
        System.out.println("Car make: " + make);
        System.out.println("Car model: " + model);
        System.out.println("Car year: " + year);
    }

    public static void main(String[] args) {
        // Using default constructor
        Car car1 = new Car();
        System.out.println("Car 1 Details:");
        car1.display();

        System.out.println();

        // Using constructor with two parameters
        Car car2 = new Car("Honda", "Civic");
        System.out.println("Car 2 Details:");
        car2.display();

        System.out.println();

        // Using constructor with three parameters
        Car car3 = new Car("Ford", "Mustang", 2022);
        System.out.println("Car 3 Details:");
        car3.display();
    }
}
```

Explanation:

1. **Default Constructor**:
   - The constructor `Car()` does not take any parameters and initializes the car with default values (`"Unknown"` for `make` and `model`, and `0` for `year`).
2. **Constructor with Two Parameters**:
   - The constructor `Car(String make, String model)` initializes the car with the given `make` and `model`. The `year` is set to a default value of `2020`.
3. **Constructor with Three Parameters**:
   - The constructor `Car(String make, String model, int year)` initializes the car with the given `make`, `model`, and `year`.
4. **Method `display()`**:
   - The `display()` method prints the car's details to the console.
5. **In the `main` method**:
   - We create three `Car` objects using different constructors:
     - `car1` uses the default constructor.

- `car2` uses the constructor with two parameters.
- `car3` uses the constructor with three parameters.

Output:

```
Car 1 Details:
Car make: Unknown
Car model: Unknown
Car year: 0

Car 2 Details:
Car make: Honda
Car model: Civic
Car year: 2020

Car 3 Details:
Car make: Ford
Car model: Mustang
Car year: 2022
```

Key Points on Constructor Overloading:

1. **Multiple Constructors**: Constructor overloading allows you to define multiple constructors with different parameter lists in the same class. Java will automatically call the appropriate constructor based on the arguments passed when creating the object.
2. **Constructor Selection**: The Java compiler selects the constructor that matches the number and types of arguments provided during object creation.
3. **Flexibility**: Constructor overloading provides flexibility in object creation. You can create an object in different ways, depending on what information you have available at the time.